ENGINEERING TRIPOS PART IIA

ELECTRICAL AND INFORMATION ENGINEERING TEACHING LABORATORY

EXPERIMENT 3B2

FPGA PROGRAMMING AND TESTING

OBJECTIVES:

- Become familiar with a typical Field Programmable Gate Array (FPGA) device, learn its features and have practice at using it.
- Examine a logic circuit and learn how to realize it in a typical CAD flow process for designing circuits that are implemented using FPGA devices.
- Configure a designed circuit into a FPGA device, test its functionality, and understand the device capabilities.

CAUTION! Misuse may damage FPGA boards. Read the instructions in this handout and apply them carefully.

1. Introduction

Field Programmable Gate Arrays (FPGAs) are found just about everywhere. From your HDTV at home to the cell phone tower in your area, to the ATM at your bank, digital logic in the form of programmable logic devices is there, doing everything from controlling how a system works, like a Central Processing Unit (CPU), to behaving like a traffic officer in high-speed switching applications required for networking and communication. But what exactly is this technology? How does it work and how can it be used in so many different applications? This experiment should help answer these questions by offering hands-on experience on programming a FPGA device, aiming at showing why programmable logic technology is so desirable to digital logic designers.

A FPGA is an integrated circuit designed to be configured by a customer or a designer after manufacturing – hence "field programmable". The FPGA configuration is generally specified using a Hardware Description Language (HDL). Circuit diagrams were previously used to specify the configuration, but this is increasingly rare. FPGAs contain an array of programmable logic blocks, and a hierarchy of reconfigurable interconnects that allow the blocks to be "wired together", like many logic gates that can be inter-wired in different configurations. Logic blocks can be configured to perform complex combinational functions, or merely simple logic gates like AND and XOR. In most FPGAs, logic blocks also include memory elements, which may be simple flip-flops (bistables or registers) or more complete blocks of memory.

The 3B2 lab offers a general overview of a typical Computer Aided Design (CAD) flow for designing circuits that are implemented by using FPGA devices, and shows students how this flow is realized. The design process is illustrated by giving step-by-step instructions for using the CAD software to implement a simple circuit into a typical FPGA device. The lab makes use of the Verilog design entry method, in which the student specifies the desired circuit in the Verilog. Verilog is a hardware description language (HDL). The last step in the design process involves configuring the designed circuit into an actual FPGA device.

It is recommended you review part 1A digital electronics. There you will find an introduction to Verilog, discussed in more detail during the 3B2 module.

Contents:

2. Apparatus

2.1. CAD software

The 3B2 lab experiment is implemented using the Quartus Prime CAD system from Altera (Intel). The Quartus software is a complete CAD system for designing digital circuits. CAD software makes it easy to implement a desired logic circuit by using a programmable logic device, such as a FPGA chip. A typical FPGA CAD flow is illustrated in Figure 1.

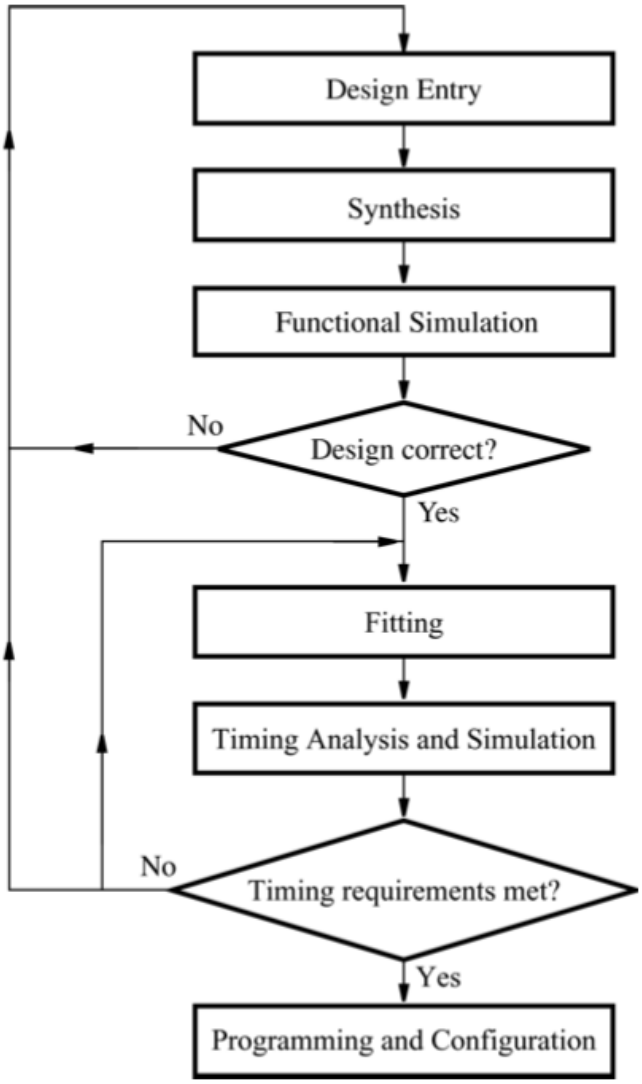

Figure 1. Typical CAD flow

The CAD flow involves the following steps:

• **Design Entry** – the desired circuit is specified either by means of a schematic diagram, or by using a hardware description language, such as Verilog

• **Synthesis** – the entered design is synthesized into a circuit that consists of the logic elements (LEs) provided in the FPGA chip

• **Functional Simulation** – the synthesized circuit is tested to verify its functional correctness; this simulation does not take into account any timing issues

• **Fitting** – the CAD Fitter tool determines the placement of the LEs defined in the netlist into the LEs in an actual FPGA chip; it also chooses routing wires in the chip to make the required connections between specific LEs

• **Timing Analysis** – propagation delays along the various paths in the fitted circuit are analysed to provide an indication of the expected performance of the circuit

• **Timing Simulation** – the fitted circuit is tested to verify both its functional correctness and timing

• **Programming and Configuration** – the designed circuit is implemented in a physical FPGA chip by programming the configuration switches that configure the LEs and establish the required wiring connections

The 3B2 experiment uses Verilog and makes use of the graphical user interface to invoke the Quartus Prime commands. All major steps needed in this experiment are described in this handout. The student will learn about:

• Creating a project

• Design entry using Verilog code

• Synthesizing a circuit specified in Verilog code

• Fitting a synthesized circuit into a FPGA

• Assigning the circuit inputs and outputs to specific pins on the FPGA

• Programming and configuring the FPGA chip

The software is installed and available in the EIETL computers. A full description of the software with plenty of tutorials (and more) is available online at Altera's University Program: https://www.altera.com/support/training/university/overview.html. For those who wish to use the software at home or PCs, a licence-free version is also available (Quartus Prime Lite Edition).

2.2. FPGA device

The last step in the design process involves configuring the designed circuit in an actual FPGA device. For this experiment, DE1-SoC development boards are available, which are connected to the EIETL computers that have Quartus Prime software installed. The DE1-SoC presents a hardware design platform built around the Altera System-on-Chip (SoC) Cyclone V FPGA, which integrates an ARM-based hard processor system (HPS) consisting of processor, peripherals and memory interfaces tied seamlessly with the FPGA fabric. The board has many features that allow students to implement a wide range of designed circuits, from simple circuits to various multimedia projects. It is important students familiarise with the DE1-SoC board User Manual.

Figure 2 gives the block diagram of the DE1-SoC board. All connections are made through the Cyclone V SoC FPGA device. This provides maximum flexibility, thus, the student can configure the FPGA to implement any system design.

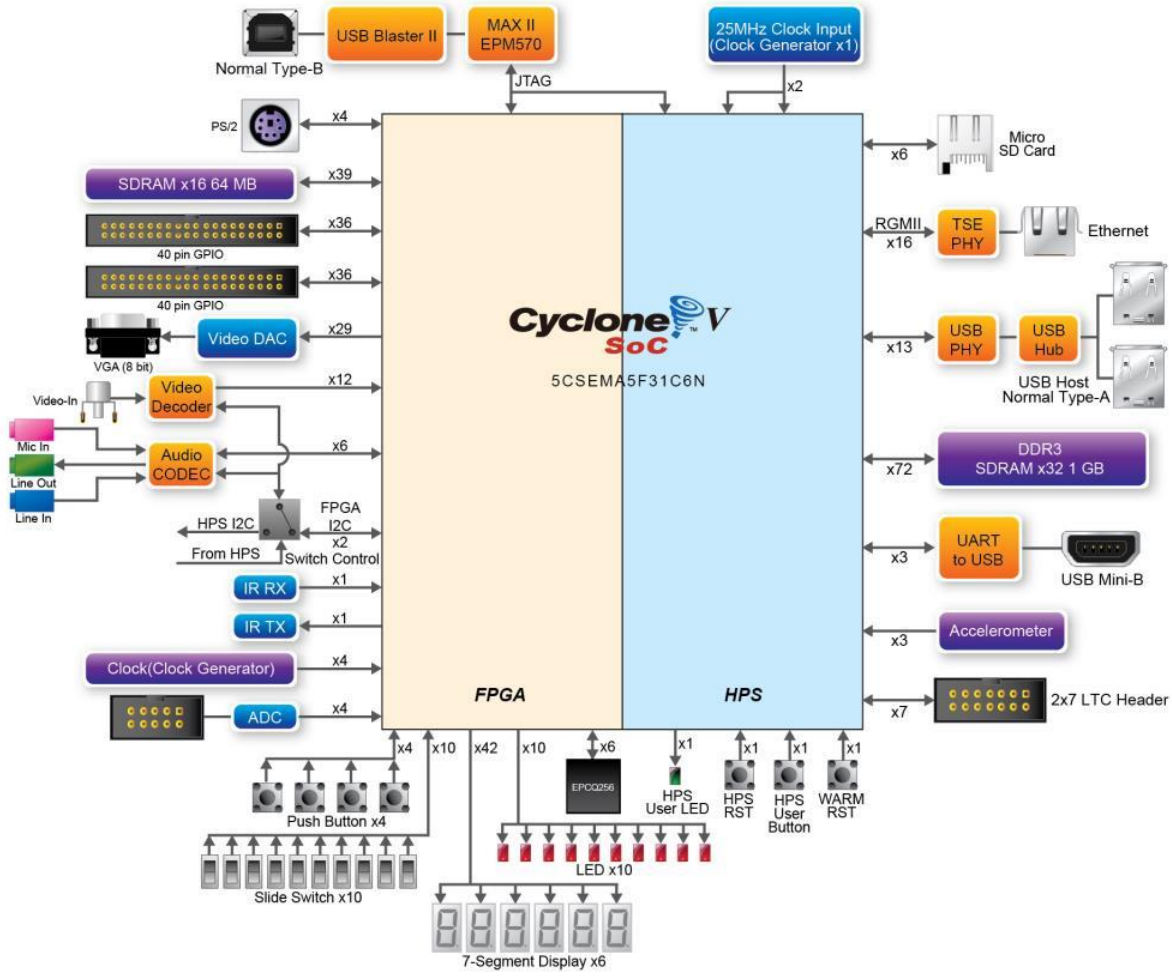

Figure 2. Block Diagram of DE1-SoC

In order to use the DE1-SoC board, the student has to be familiar with the Quartus software.

3. Starting a new project

Each logic circuit being designed with Quartus Prime software is called a project. The software works on one project at a time and keeps all information for that project in a single directory (folder) in the file system. To begin a new logic circuit design, the first step is to create a directory to hold its files. The running example for this experiment is a simple circuit for a traffic light controller (tlc).

To start working on a new design you first have to define a new design project. Start the Quartus Prime software. Create a new project as follows:

3.1. Select **File** > **New Project Wizard** and click **Next**, which asks for the name and directory of the project.

3.2. Set the working directory of your choice, e.g., **Experiment_3B2**. The project must have a name, which is usually the same as the top-level design unit that will be included in the

project. Choose **tlc** as the name for both the project and the top-level design unit, as shown in Figure 3. Press **Next**, then click **Yes** to create directory.

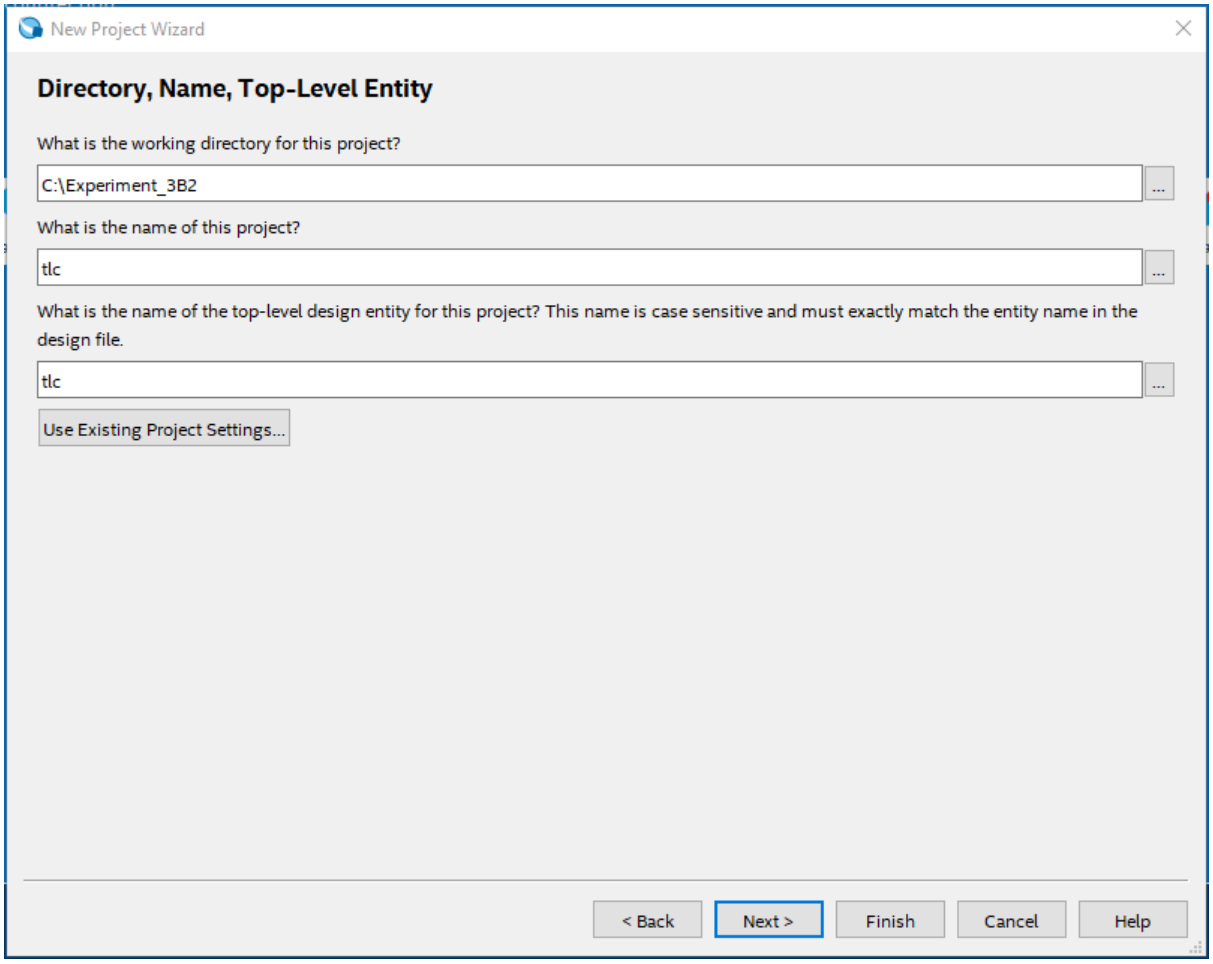

Figure 3. Creation of a new project

3.3. Choose **Empty project** in the **Project Type** window.

3.4. Click **Next** in the **Add Files** window.

3.5. You have to specify the type of device in which the designed circuit will be implemented. From the list of available devices, choose the Cyclone V device family and then select your device in the family (e.g. Altera Cyclone V SE 5CSEMA5F31C6 device). Press **Next**.

3.6. Click **Next** in the **EDA Tool Settings** window.

3.7. Click **Finish** in the **Summary** window.

4. Verilog design entry

In this experiment, we want to implement a simple controller for a pedestrian light-controlled crossing, as shown in Figure 4.

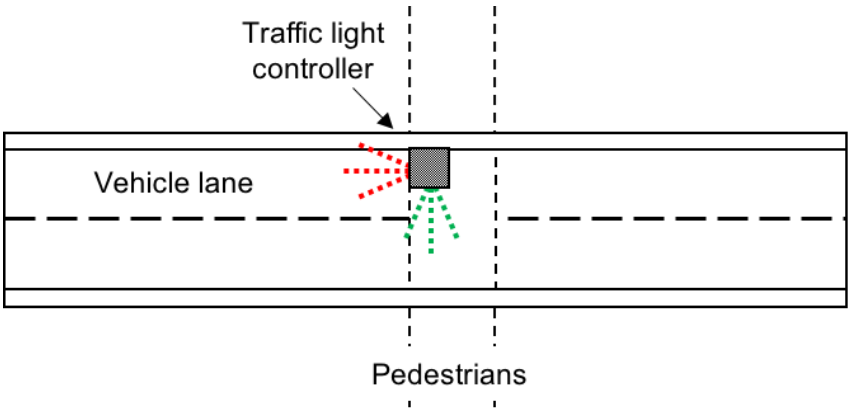

Figure 4. Pedestrian light-controlled crossing

The requirements for the traffic light controller (tlc) are:

I. The initial state is vehicle GREEN and pedestrian RED.
II. It can only change state by demand, i.e. a pedestrian request or reset.
III. After a request is made, the following 3-state sequence needs to be implemented:
   a. It starts by turning vehicle to YELLOW for 5 seconds;
   b. Then it turns vehicle to RED and pedestrian to GREEN at the same time, both for 10 seconds;
   c. Then it returns to the initial state.
IV. A reset command returns the system to its initial state.

A simple state diagram for the logic circuit described above is shown in Figure 5, where states G, Y and R stand for GREEN, YELLOW and RED respectively.

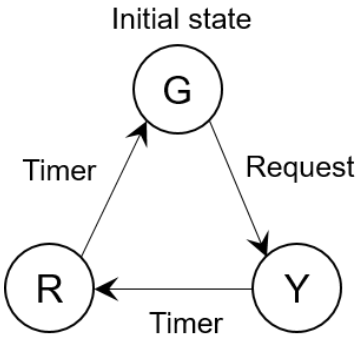

Figure 5. Simple state diagram for traffic light controller

Please note, the traffic lights (states) in Figure 5 are the vehicle lights. GREEN light for vehicles means RED for pedestrians, etc. A state table can also be written to include next states, as well as inputs and outputs, as shown in Table 1 below.

State	**Next state**				**Vehicle lights**			**Pedestrian lights**	
	Reset	Request	5 s	10 s	GREEN	YELLOW	RED	GREEN	RED
G	G	Y			1	0	0	0	1
Y	G		R		0	1	0	0	1
R	G			G	0	0	1	1	0

Table 1. State table for traffic light controller

The required circuit needs to be described in Verilog code, in order to be implemented in a FPGA device.

Note: The Quartus software also allows you to define the desired circuit in the form of a schematic diagram. To do this you will need to expand the state table to include excitation columns for bistables and draw the circuit diagram. Although using schematic diagrams is an excellent exercise to get started with the basics, it is rarely used in high-end design and does not showcase the potential of using HDLs such as Verilog. Verilog can be used as higher level language with behavioural synthesis support. By simply describing the functionality of a circuit and letting the synthesis tool build it for you, you can produce very powerful functions in just a few lines of code. Doing this using a structural language, is more like programming in low level assembly with the circuit having a very specific implementation, so changes to that implementation do not scale easily and require much more effort.

A Verilog code example is given in Figure 6 below. You may use your own. Note that the top-level Verilog module is called **tlc** to match the name given in Figure 3, which was specified when the project was created. This code can be typed into a file by using any text editor, or by using the Quartus Prime text editing facilities. The file name must include the extension **v**, which indicates a Verilog file.

Note: Before writing the Verilog code, it is important to understand the features available on your DE1-SoC board. For this experiment, you need a clock to time the states of your circuit on the order of seconds. The DE1-SoC board has an integrated 50 MHz clock signal available, but this is orders of magnitude faster than the time events required in traffic lights. A way to turn it into a suitable clock is to use counters or dividers as seen in 1A. These can be written in Verilog in a few lines.

```
module tlc (
    input  wire       clk,
    input  wire       request,
    input  wire       reset,
    output reg  [4:0] \output
);

    localparam G = 2'd0;
    localparam Y = 2'd1;
    localparam R = 2'd2;

    reg [1:0]  state;
    reg [28:0] count;

    always @(posedge clk or negedge reset) begin
        if (!reset) begin
            state <= G;
            count <= 29'd0;
        end else begin
            case (state)
                G: begin
                    if (request == 1'b0) begin
                        state <= Y;
                        count <= 29'd0;
                    end
                end
                Y: begin
                    if (count == 29'd250000000) begin
```

```
                    state <= R;
                    count <= 29'd0;
                end else begin
                    count <= count + 29'd1;
                end
            end
            R: begin
                if (count == 29'd500000000) begin
                    state <= G;
                    count <= 29'd0;
                end else begin
                    count <= count + 29'd1;
                end
            end
            default: begin
                state <= G;
                count <= 29'd0;
            end
        endcase
    end
end

always @(*) begin
    case (state)
        G: \output = 5'b10001;
        Y: \output = 5'b01001;
        R: \output = 5'b00110;
        default: \output = 5'b10001;
    endcase
end

endmodule
```

Figure 6. Verilog code for the circuit in Figure 5

4.1. Using the Quartus text editor

Select **File** > **New**, choose **Verilog HDL File**, and click **OK**. This opens the Text Editor window. The first step is to specify a name for the file that will be created. Select **File** > **Save As**, choose **Verilog HDL File**. In the box labelled **File name** type **tlc.v**. Put a checkmark in the box **Add file to current project**. Click **Save**, which puts the file into the project directory, e.g. **Experiment_3B2**. Enter the Verilog code in Figure 6 or your own into the Text Editor and save the file by typing **File** > **Save**, or by typing the shortcut Ctrl-s.

Most of the commands available in the Text Editor are self-explanatory. Text is entered at the insertion point, which is indicated by a thin vertical line. The insertion point can be moved either by using the keyboard arrow keys or by using the mouse. Two features of the Text Editor are especially convenient for typing Verilog code. First, the editor can display different types of Verilog statements in different colours, which is the default choice. Second, the editor can automatically indent the text on a new line so that it matches the previous line.

5. Compiling the designed circuit

The Verilog code in the file tlc.v is processed by several Quartus Prime tools that analyse the code, synthesize the circuit, and generate an implementation of it for the target chip. These tools are controlled by the application program called the Compiler.

Run the Compiler by selecting **Processing** > **Start Compilation**, or by clicking on the toolbar icon  that looks like a blue triangle. Your project must be saved before compiling. As the compilation moves through various stages, its progress is reported in a window on the left side of the Quartus Prime display. In the message window, at the bottom of the figure, various messages are displayed throughout the compilation process. In case of errors, there will be appropriate messages given.

When the compilation is finished, a compilation report is produced. A tab showing this report is opened automatically. The tab can be closed in the normal way, and it can be opened at any time either by selecting **Processing** > **Compilation Report** or by clicking on the icon  . The report includes a number of sections listed on the left side. For the circuit in Figure 6, the Compiler Flow Summary indicates that 31 logic elements, 35 registers and 8 pins are needed to implement this circuit on the selected FPGA chip.

6. Pin assignment

During the compilation above, the Quartus Prime Compiler was free to choose any pins on the selected FPGA to serve as inputs and outputs. However, the DE1-SoC board has hardwired connections between the FPGA pins and the other components on the board. You will need to provide inputs for the **request** and **reset**, which can be push-buttons, as well as locate the input for **clk**, and connect the **output** to light-emitting diodes (LEDs) on your DE1 board. The FPGA pin assignment for the DE1-SoC can be found in the User Manual.

Note: The Compiler may automatically assign a clock signal to **clk**. However, it is a good exercise to locate the clock signal pin on your board and make sure it is assigned properly. The **output** in Figure 6 is a vector which contains all 5 LED outputs: 3 for the vehicle and 2 for the pedestrian, i.e. **output[4], output[3], …** to **output[0]**.

Pin assignments are made by using the Assignment Editor. Select **Assignments** > **Assignment Editor**. In the **Category** drop-down menu select **All**. Click on the **<<new>>** button located near the top left corner to make a new item appear in the table. Double click the box under the column labelled **To** so that the **Node Finder** button appears, and Click on it (not the drop down arrow). In the **Filter** drop-down menu select **Pins: all**. Then click the **List** button to display the input and output pins to be assigned: **request**, **reset**, **clk** and **output**. Click on **request** as the first pin to be assigned and click the **>** button; this will enter **request** in the **Selected Nodes** box. Click **OK**. **request** will now appear in the box under the column labelled **To**. Alternatively, the node name can be entered directly by double-clicking the box under the **To** column and typing in the node name.

Follow this by double-clicking on the box to the right of this new **request** entry, in the column labelled **Assignment Name**. Now, in the drop-down menu scroll down and select **Location (Accepts wildcards/groups)**. Finally, double-click the box in the column labelled **Value**.

Type the pin assignment corresponding to the push-button you want to use for **request** for your DE1 board, listed in the User Manual.

Use the same procedure to assign inputs **reset** and **clk**, and the outputs **output** to the appropriate pins listed in DE1-SoC User Manual. To save the assignments made, choose **File** > **Save**. Recompile the circuit, so that it will be compiled with the correct pin assignments. An example using the DE1-SoC board is shown in Figure 7.

Note: The DE1-SoC has ten user-controllable LEDs connected to the FPGA on the board, but they are all RED. To implement the traffic light controller, using the LEDs available on the board, RED LEDs must also be assigned to the controller GREEN and YELLOW lights. Alternatively, GREEN and YELLOW LEDs can be connected through the two 40-pin expansion headers (GPIO ports). The header connects directly to 36 pins of the Cyclone V SoC FPGA, and also provides DC +5V (VCC5), DC +3.3V (VCC3P3), and two GND pins.

N.B. The LED assignments (all RED on the board) for the example in Figure 7, are:

LEDR0 – pedestrian RED

LEDR2 – pedestrian GREEN

LEDR4 – vehicle RED

LEDR7 – vehicle YELLOW

LEDR9 – vehicle GREEN

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔		in request	Location	PIN_AA15	Yes			
2	✔		in reset	Location	PIN_AA14	Yes			
3	✔		out output[0]	Location	PIN_V16	Yes			
4	✔		out output[1]	Location	PIN_V17	Yes			
5	✔		out output[2]	Location	PIN_W17	Yes			
6	✔		out output[3]	Location	PIN_W20	Yes			
7	✔		out output[4]	Location	PIN_Y21	Yes			
8	✔		in clk	Location	PIN_AF14	Yes			
9		<<new>>	<<new>>	<<new>>					

Figure 7. The complete assignment

7. Programming and configuring the FPGA device

The FPGA device must be programmed and configured to implement the designed circuit. The required configuration file is generated by the Quartus Prime Compiler's Assembler module. The configuration to be done in two different ways, known as JTAG and AS modes. The configuration data is transferred from the host computer (which runs the Quartus Prime software) to the board by means of a cable that connects a USB port on the host computer to the USB-Blaster connector on the board. To use this connection, it is necessary to have the USB-Blaster driver installed. The driver is already installed in the EITL computers. For information about installing the driver, consult the tutorial Getting Started with Intel's DE-

Series. Before using the board, make sure that the USB cable is properly connected and turn on the power supply switch on the board.

In the JTAG mode, the configuration data is loaded directly into the FPGA device. The acronym JTAG stands for Joint Test Action Group. This group defined a simple way for testing digital circuits and loading data into them, which became an IEEE standard. If the FPGA is configured in this manner, it will retain its configuration as long as the power remains turned on. The configuration information is lost when the power is turned off. The second possibility is to use the Active Serial (AS) mode. In this case, a configuration device that includes some flash memory is used to store the configuration data. Quartus Prime software places the configuration data into the configuration device on the DE-series board. Then, this data is loaded into the FPGA upon power-up or reconfiguration. Thus, the FPGA need not be configured by the Quartus Prime software if the power is turned off and on. The choice between the two modes is made by switches on the DE-series board. Consult your manual for the location of this switch on your DE-series board. The boards should be set to JTAG mode by default. This experiment uses only the JTAG programming mode.

For the DE1-SoC board, there are two devices (FPGA and HPS) on the JTAG Chain, and the following steps should be used for programming. Select **Tools** > **Programmer**. Here it is necessary to specify the programming hardware and the mode that should be used. If not already chosen by default, select **JTAG** in the **Mode** box. Also, if the **DE-SoC** is not chosen by default, press the **Hardware Setup...** button and select the **DE-SoC** in the window that pops up.

Observe that the configuration file **tlc.sof** is listed in the **Programmer** window. If the file is not already listed, then click **Add File** and select it. It is located in the **\Experiment_3B2\output_files** folder. This is a binary file produced by the Compiler's Assembler module, which contains the data needed to configure the FPGA device. The extension .sof stands for SRAM Object File. Ensure the **Program/Configure** check box is ticked. This setting is used to select the FPGA in the Cyclone V SoC chip for programming. If the **SOCVHPS** (HPS) device is not shown as in Figure 8, click **Add Device** > **SoC Series V** > **SOCVHPS** then click **OK**. Ensure that your device order is consistent with Figure 8 by clicking on a device and then clicking **Up** or **Down**.

Now, press **Start** in the **Programmer** window. An LED on the DE1-SoC board will light up while the FPGA device is being programmed. If you see an error reported by Quartus Prime software indicating that programming failed, then check to ensure that the board is properly powered on.

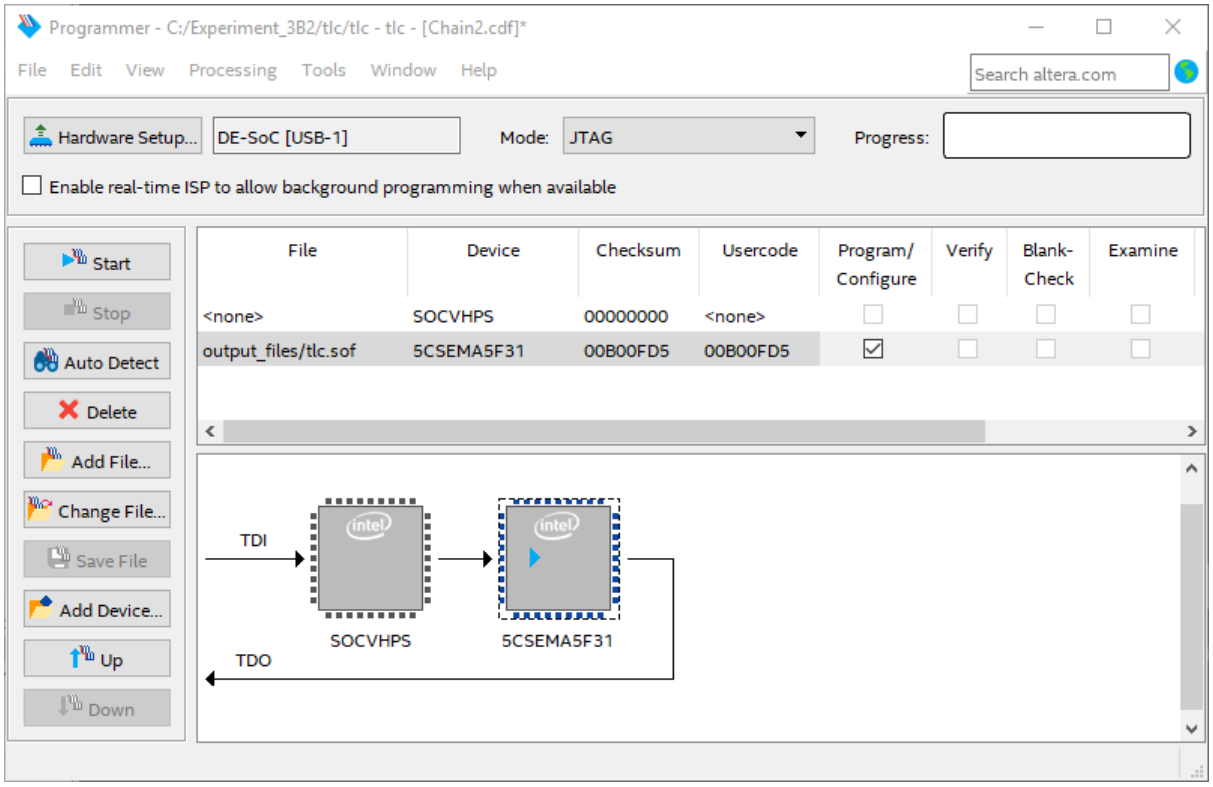

Figure 8. The Programmer window

8. Testing the designed circuit

Having downloaded the configuration data into the FPGA device, you can now test the implemented circuit. After configuration, the circuit should have a GREEN LED on for the vehicle and a RED LED on for the pedestrian. All other LEDs should be off. Try the push-button corresponding to **request**. This should turn vehicle GREEN LED off and vehicle YELLOW LED on for 5 seconds, and then turn vehicle RED LED on and pedestrian GREEN LED on, etc. Try all valuations of the input variables **request** and **reset**. Verify that the circuit implements the state table in Table 1.

If you want to make changes in the designed circuit, first close the **Programmer** window. Then make the desired changes in the Verilog design file, compile the circuit, and program the board as explained above.

9. Improving the circuit

The circuit in Figure 6 can be improved. Please note, continuous pedestrian request may result in blocking the vehicle lane (RED light). To account for this, the controller should complete a pedestrian request at a time, and after a request is completed, it should allow for, let's say, 10 seconds of vehicle GREEN before taking up a second request. This is the case with traffic light controllers.

I. Write Verilog code to implement your design

II. Compile the circuit

III. Download the circuit into your FPGA board and test it

Options

Note: Options are not compulsory. You may attempt them if you finish the experiment earlier.

A. Some traffic light controllers also display the time left before changing state. Design and implement a circuit on your DE board that acts as a timer. It should display the seconds (from 10 to 0) on two 7-segment displays, e.g. HEX1 and HEX0. Stop the timer whenever *KEY*0 is being pressed. Add the timer to your circuit to display the time left for each state.

B. In this option you are to implement a Morse-code encoder using a finite state machine (FSM). The Morse code uses patterns of short and long pulses to represent a message. Each letter is represented as a sequence of dots (a short pulse), and dashes (a long pulse). For example, the first eight letters of the alphabet have the following representation:

A	● —
B	— ●●●
C	— ● — ●
D	— ●●
E	●
F	●● — ●
G	— — ●
H	●●●●

Design and implement a Morse-code encoder circuit using an FSM. Your circuit should take as input one of the first eight letters of the alphabet and display the Morse code for it on a red LED. Use switches *SW*2-0 and pushbuttons *KEY*1-0 as inputs. When a user presses *KEY*1, the circuit should display the Morse code for a letter specified by *SW*2-0 (000 for A, 001 for B, etc.), using 0.5-second pulses to represent dots, and 1.5-second pulses to represent dashes. Pushbutton *KEY*0 should function as an asynchronous reset. A high-level schematic diagram of a possible circuit for the Morse-code encoder is shown in Figure 9.

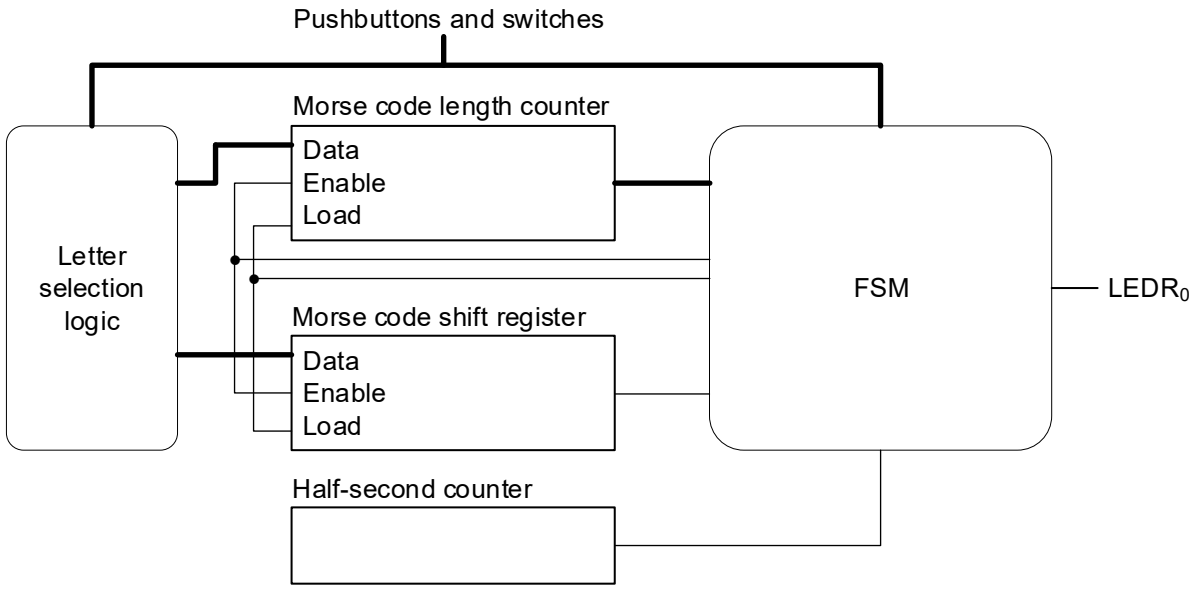

Figure 9. High-level schematic diagram of the circuit for Option B

10. Conclusions and write-up

This experiment requires a short report as write-up. Please follow the general guidelines for Part IIA reports:

https://www.vle.cam.ac.uk/course/view.php?id=72251

as well as the specific guidelines below.

Give a concise and clear record of the practical work and data recorded, together with a data discussion and analysis that highlight your understanding (data is everything you observe). In the interests of conciseness, there is no need to provide an introduction section, aims and objectives.

Although the emphasis is on a succinct report, a good write-up should clearly explain:

- what each part of the experiment sought to explore;
- what you found out, and what precautions you needed; and
- brief conclusions about each main result.

Your print-outs or photos should be attached as an appendix. A concise write-up should be ideal to revise from at exam time.

Hint: …you have been sent, by your company, to test a new technology. As a result, you are responsible to provide the company board with a report, based on which a decision will be made on whether to adopt the technology or not. The decision will impact the future of the company, and yours…